
Computer Architecture

Mr. Muhammad Adeel

W-15

Course Logistics

- 2 Lectures per week
- Quizzes: (6 quizzes): 20% **(03 Quizzes done)**
- Assignments: (2): 10% **(10 Abs decided)**
- Midterm exam (M1 + M2): 25% **(25 decided)**
- Final exam : 40%
- CP: 5%
- No quiz retakes will be entertained.
- Late assignments will be marked zero.
- Make sure to follow NU attendance policy.

Post-M2-Plan

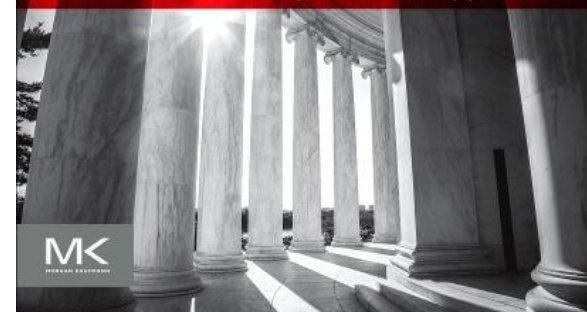
- **Advanced Microarchitecture (W12)**
 - Deep Pipelines
- **Memory Systems (W13-W15)**
 - Understanding Memory Systems
 - Cache
 - Virtual Memory
- **GPU Architecture (If time permits)**

Digital Design and Computer Architecture RISC-V Edition



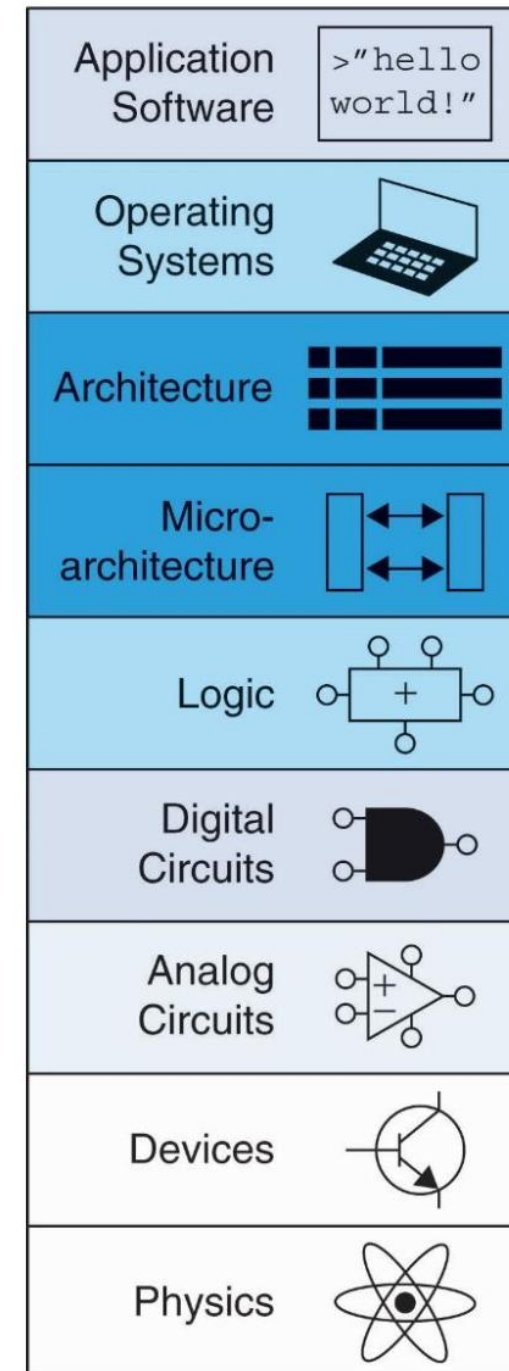
Sarah L. Harris
David Money Harris

COMPUTER ARCHITECTURE A Quantitative Approach



Outline

- Introduction
- Memory System Performance Analysis
- Caches
- Virtual Memory

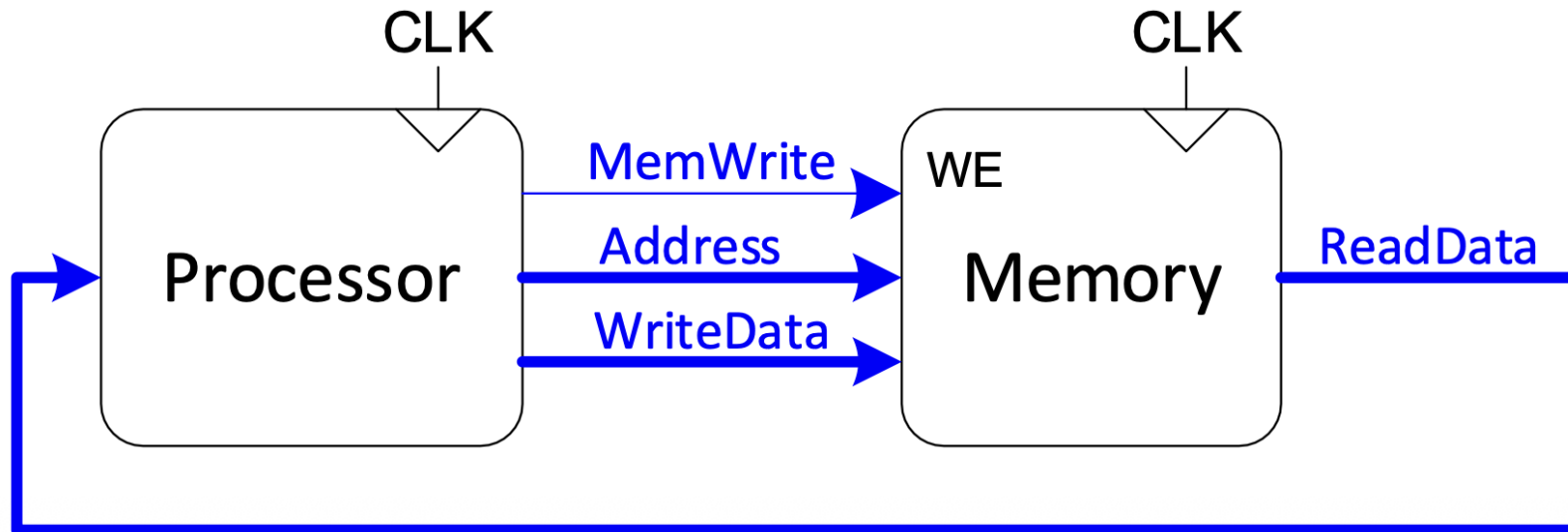


Introduction

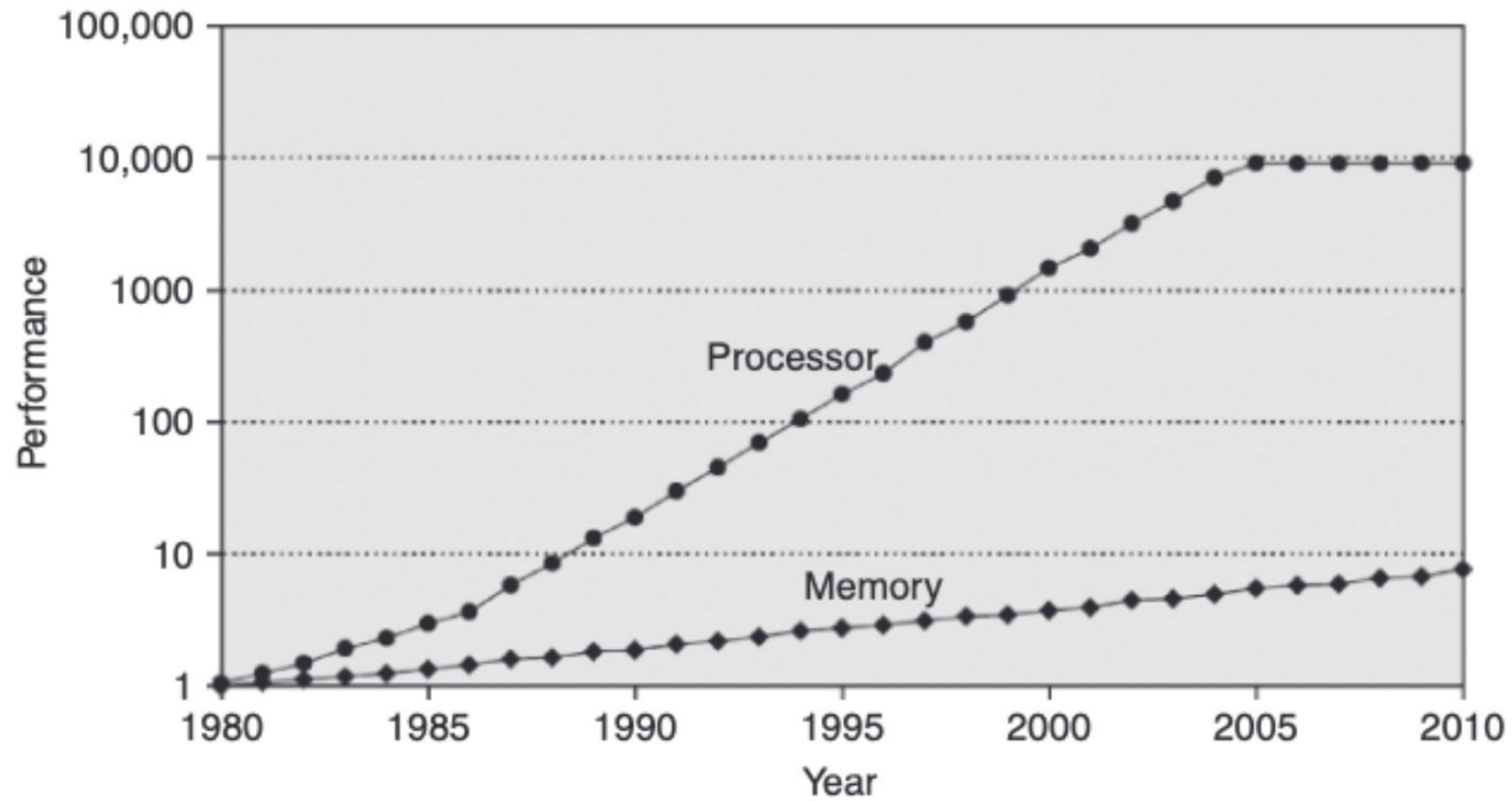
- **Computer performance depends on:**
 - **Processor** performance
 - **Memory system** performance

Processor / Memory Interface:

The processor sends an address over the Address bus to the memory system. For a read, Mem Write is 0 and the memory returns the data on the ReadData bus. For a write, Mem Write is 1 and the processor sends data to memory on the WriteData bus.



Performance Gap

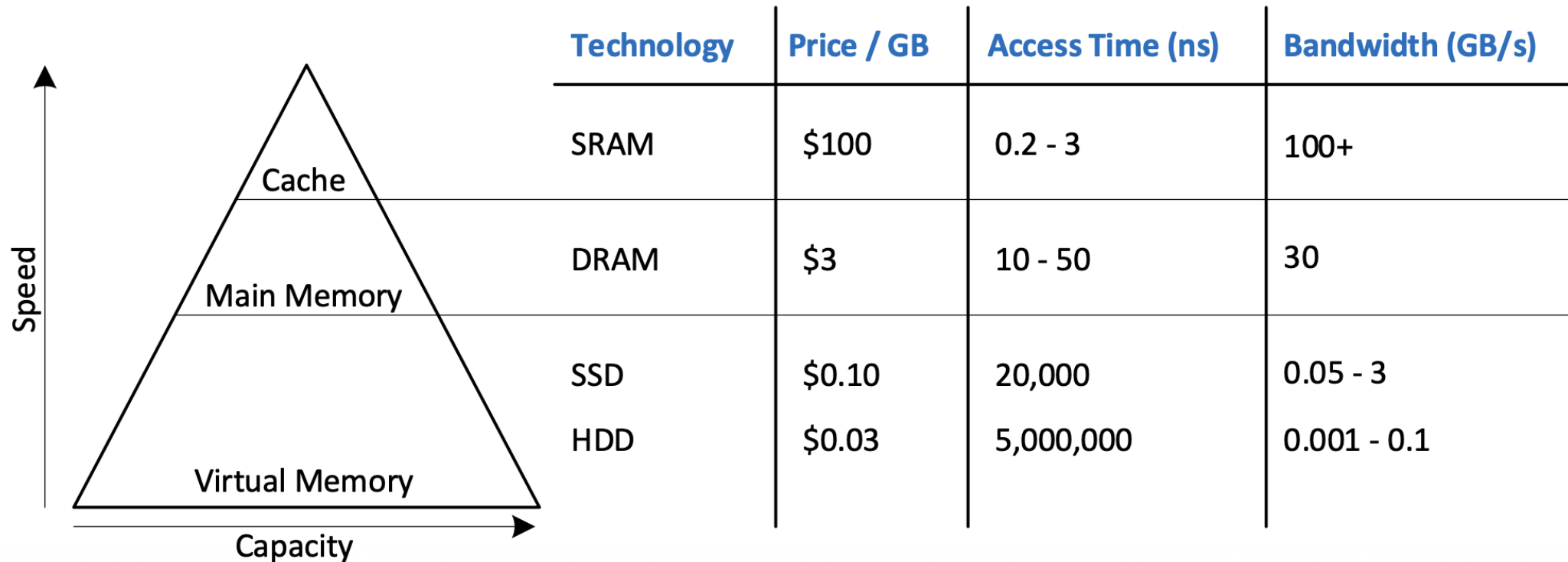
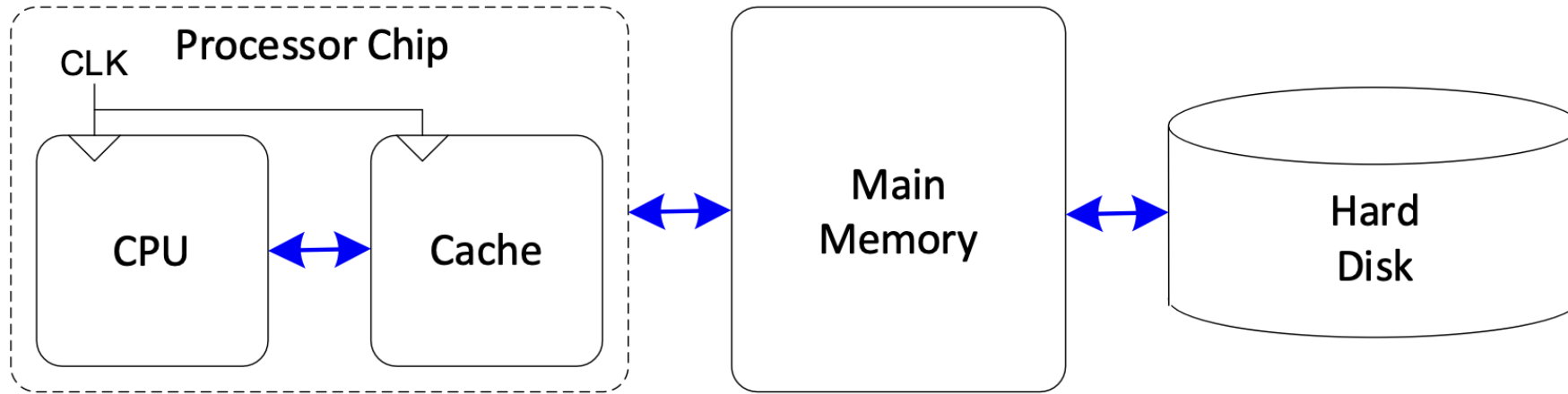


Memoray System Challenges

- Make memory system appear as fast as processor
- Use hierarchy of memories
- Ideal memory:
 - Fast
 - Cheap (inexpensive)
 - Large (capacity)
- But can only choose two!



Memoray Hierarchy



Locality

Exploit locality to make memory accesses fast:

- **Temporal Locality:**

- Locality in time
- If data used recently, likely to use it again soon
- **How to exploit:** keep recently accessed data in higher levels of memory hierarchy

- **Spatial Locality:**

- Locality in space
- If data used recently, likely to use nearby data soon
- **How to exploit:** when access data, bring nearby data into higher levels of memory hierarchy too

Memory Performance

- **Hit:** data found in that level of memory hierarchy
- **Miss:** data not found (must go to next level)

Hit Rate = # hits / # memory accesses
 = 1 – Miss Rate

Miss Rate = # misses / # memory accesses
 = 1 – Hit Rate

- **Average memory access time (AMAT):** average time for processor to access data

AMAT = $t_{\text{cache}} + MR_{\text{cache}}[t_{MM} + MR_{MM}(t_{VM})]$

Memory Performance

Memory Performance Example 2

- Suppose processor has 2 levels of hierarchy: cache and main memory
- $t_{\text{cache}} = 1$ cycle, $t_{\text{MM}} = 100$ cycles
- What is the **AMAT** (average memory access time) of the program from Example 1?

$$\begin{aligned}\text{AMAT} &= t_{\text{cache}} + MR_{\text{cache}}(t_{\text{MM}}) \\ &= [1 + 0.375(100)] \text{ cycles} \\ &= \mathbf{38.5 \text{ cycles}}\end{aligned}$$

Memory Performance Example 1

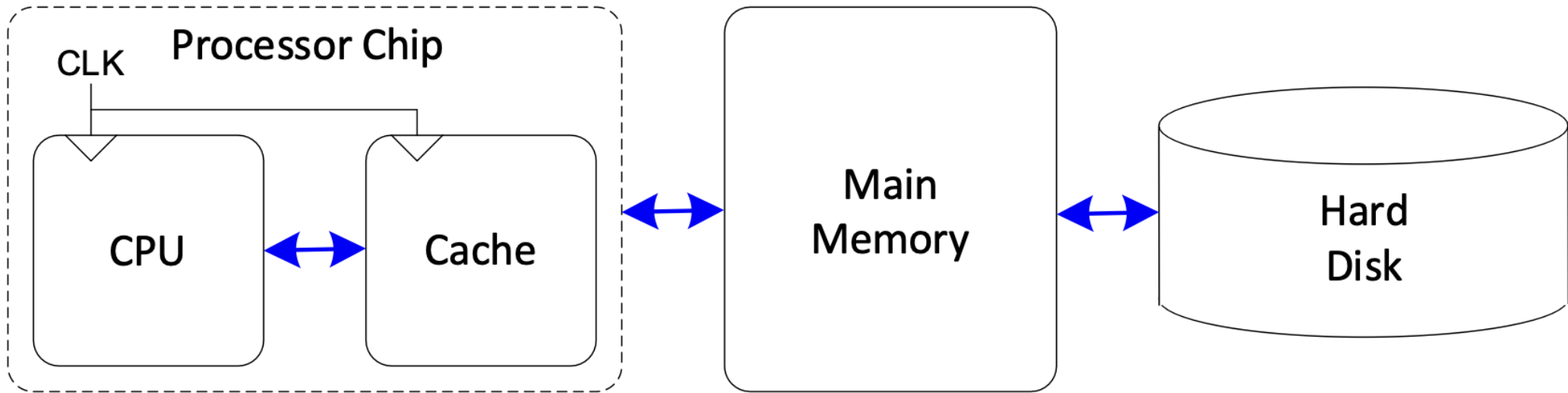
- A program has 2,000 loads and stores
- 1,250 of these data values in cache
- Rest supplied by other levels of memory hierarchy
- What are the **cache hit and miss rates**?

$$\text{Hit Rate} = 1250/2000 = \mathbf{0.625}$$

$$\text{Miss Rate} = 750/2000 = \mathbf{0.375} = 1 - \text{Hit Rate}$$

Cache

- Highest level in memory hierarchy
- Fast (typically ~ 1 cycle access time)
- Ideally supplies most data to processor
- Usually holds most recently accessed data



Cache Design Questions

- What data is held in the cache?
- How is data found?
- What data is replaced?

We will use loads as a default example but stores will also the same procedure!!!

What data is held in cache

- Ideally, cache anticipates needed data and puts it in cache
- But impossible to predict future
- Use past to predict future – temporal and spatial locality:
 - **Temporal locality**: copy newly accessed data into cache
 - **Spatial locality**: copy neighboring data into cache too

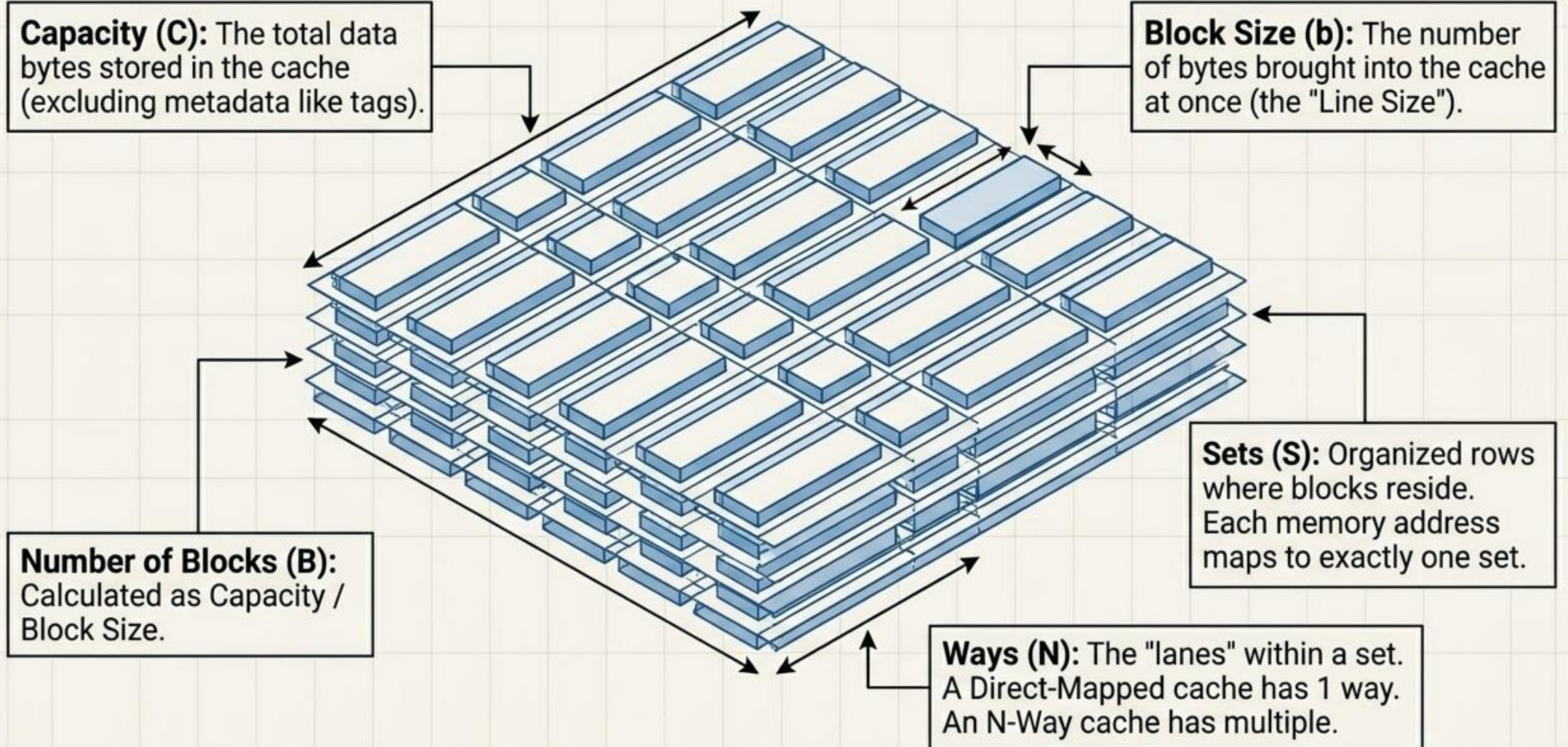
Cache Terminology

- **Capacity (C):**
 - number of data bytes in cache
- **Block size (b):** Also known as line size
 - bytes of data brought into cache at once
- **Number of blocks ($B = C/b$):**
 - number of blocks in cache: $B = C/b$
- **Degree of associativity (N):**
 - number of blocks in a set
- **Number of sets ($S = B/N$):**
 - each memory address maps to exactly one cache set

How is data found?

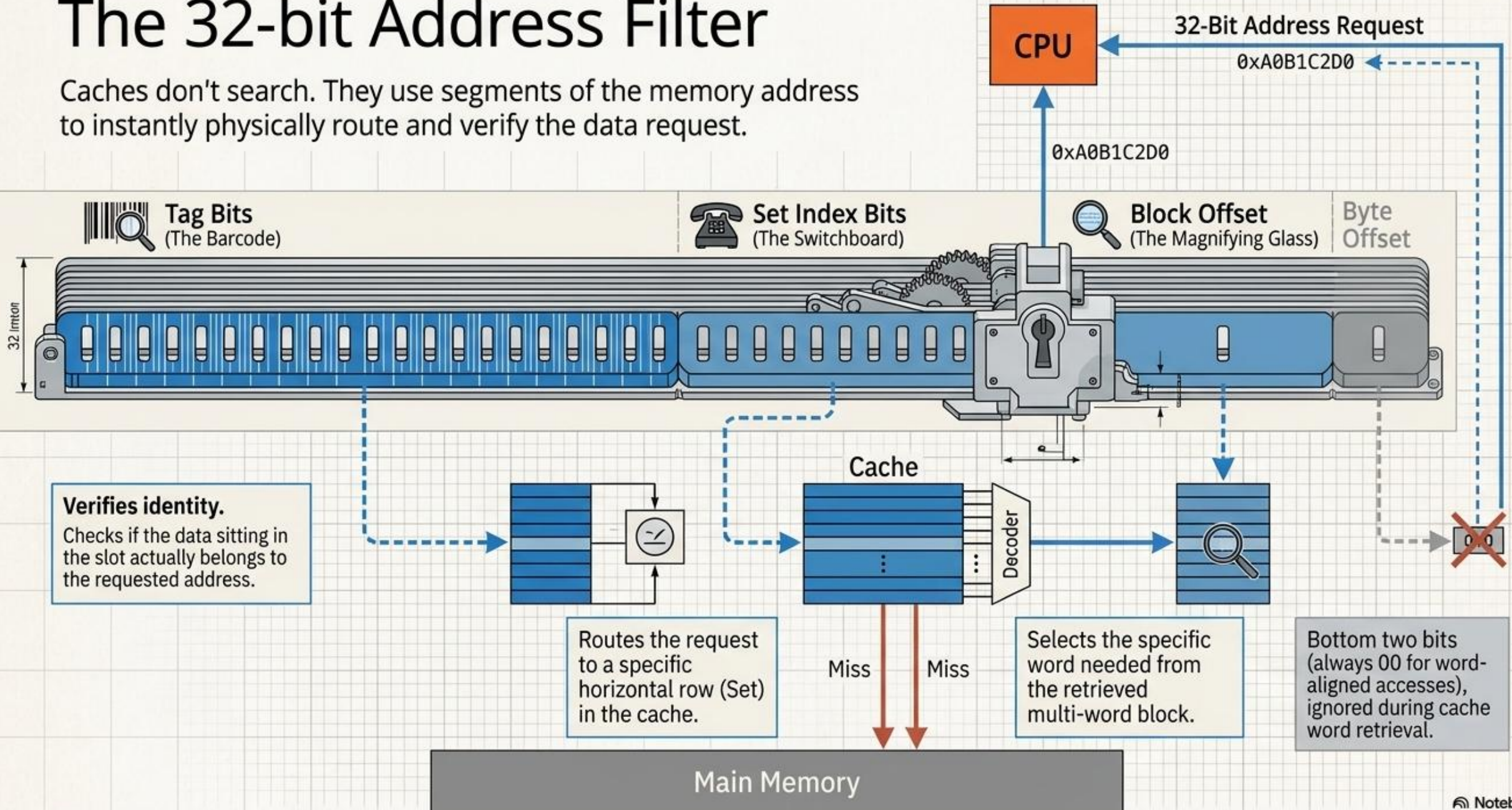
- Cache organized into S sets
- Each memory address maps to exactly one set
- Caches categorized by # of blocks in a set:
 - **Direct mapped:** 1 block per set
 - **N -way set associative:** N blocks per set
 - **Fully associative:** all cache blocks in 1 set
- Examine each organization for a cache with:
 - Capacity ($C = 8$ words)
 - Block size ($b = 1$ word)
 - So, number of blocks ($B = 8$)

Cache Architecture Definitions

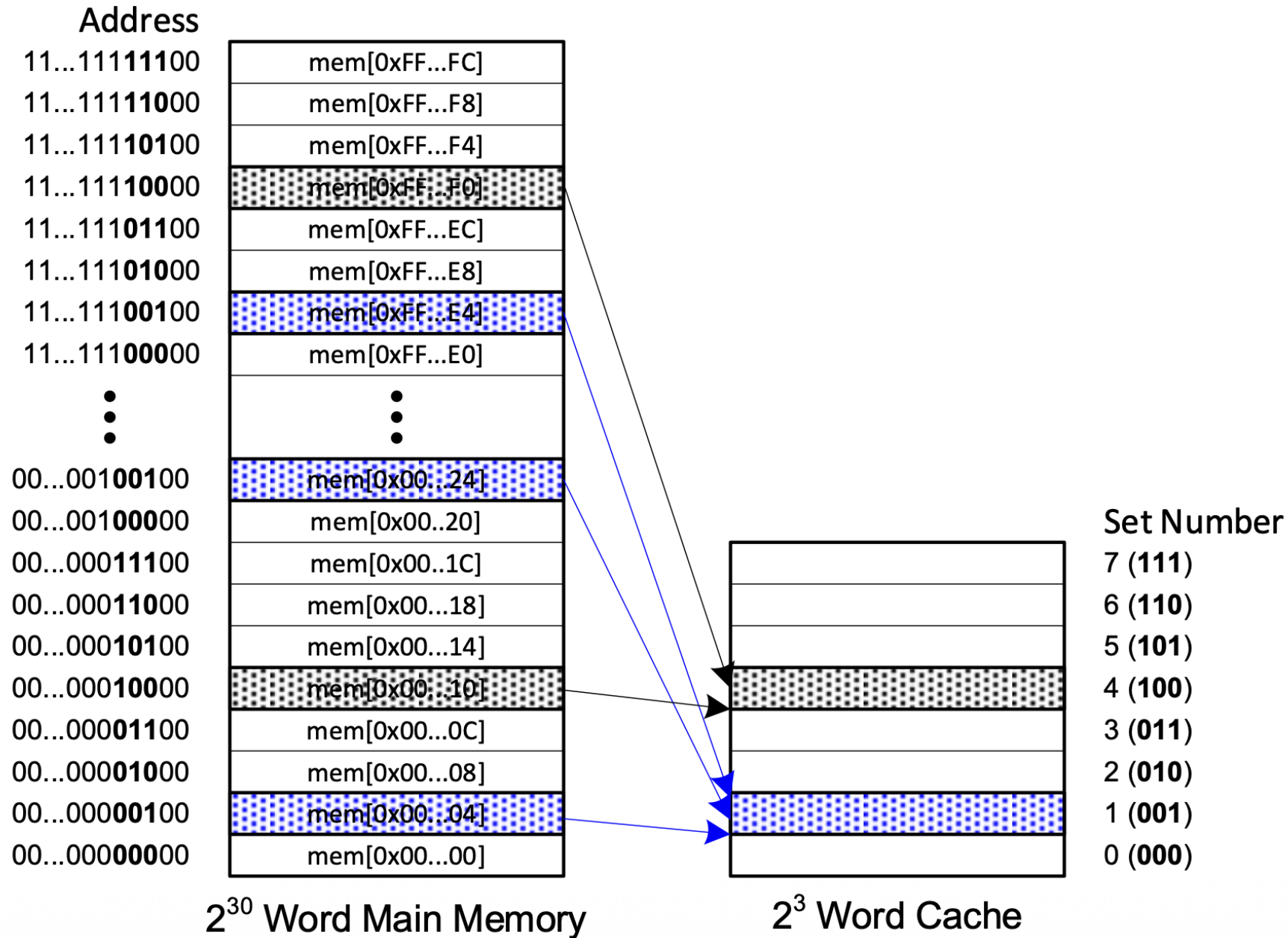


The 32-bit Address Filter

Caches don't search. They use segments of the memory address to instantly physically route and verify the data request.

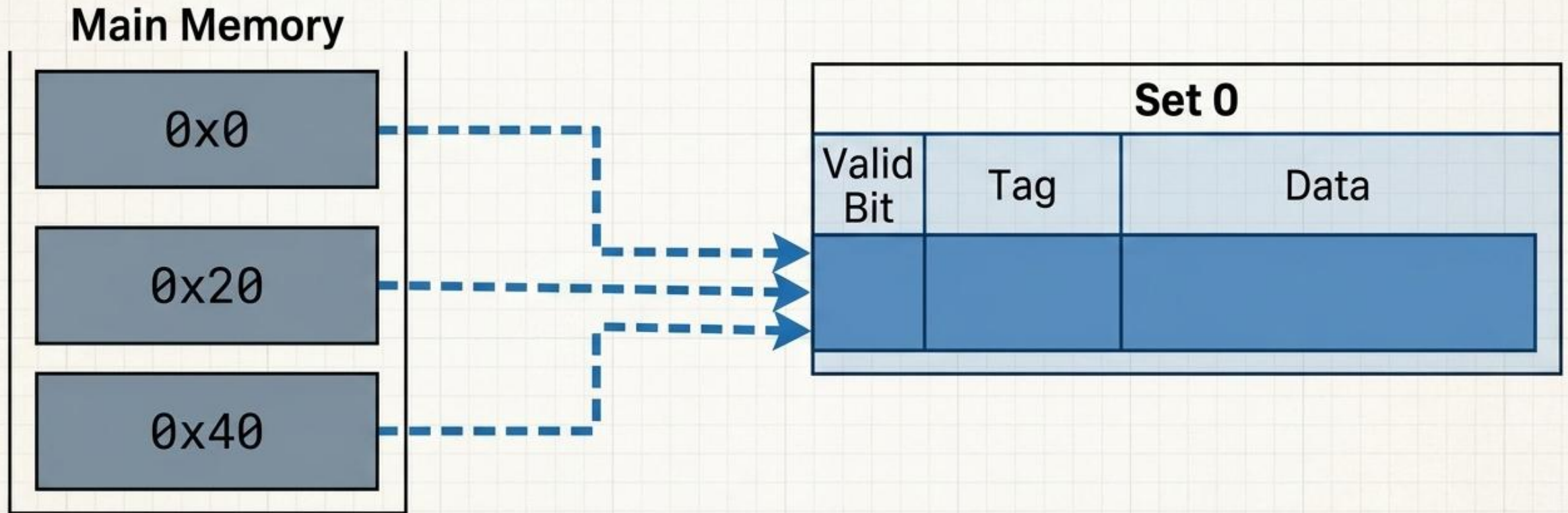


Direct Mapped Cache

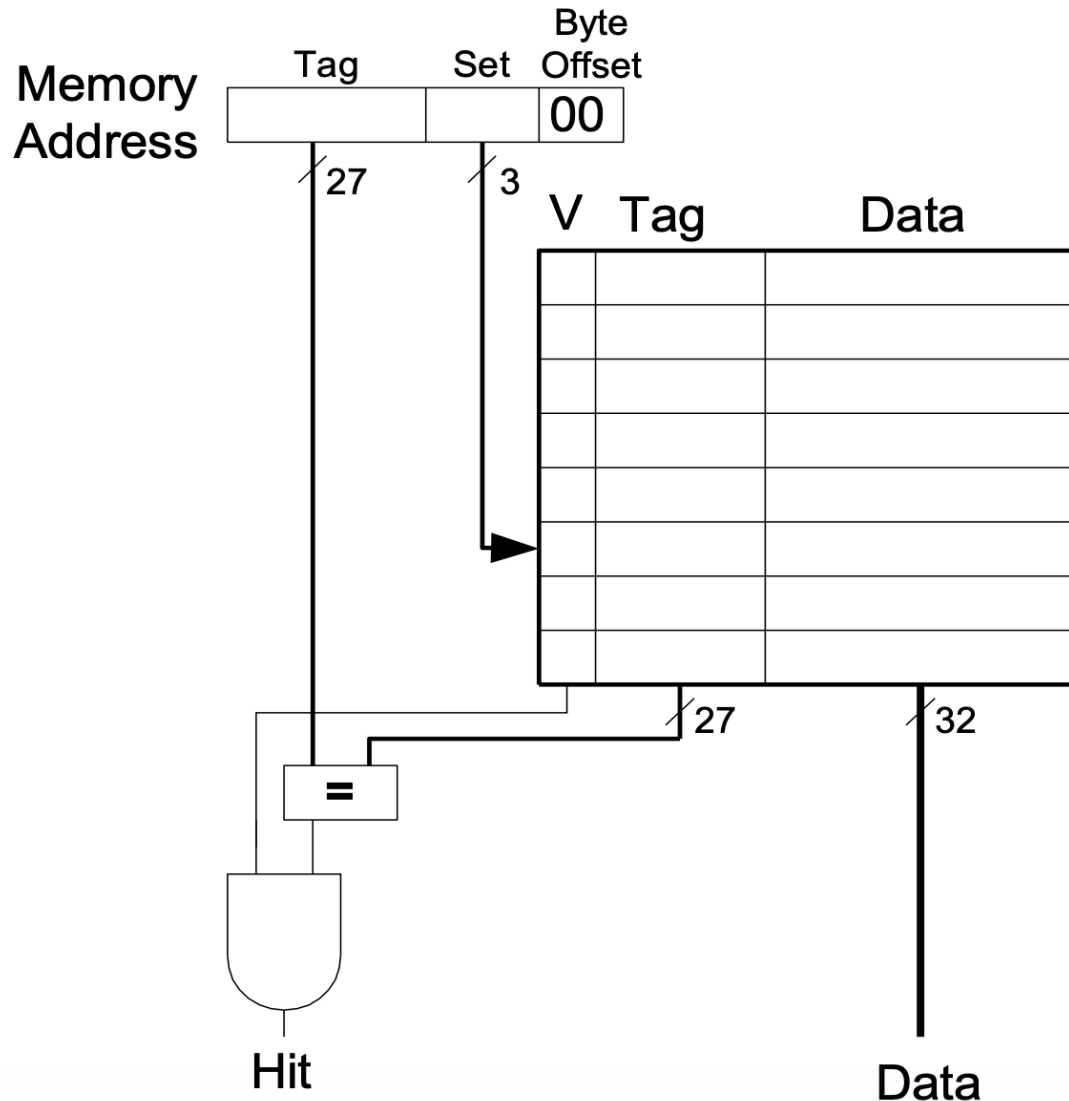


Direct-Mapped Caches route rigidly

In a Direct-Mapped cache, each memory address corresponds to exactly one block (Way) in one Set. It is incredibly fast to locate data, but highly vulnerable to traffic jams if two active variables map to the exact same spot.



Direct Mapped Cache Hardware



Because many addresses map to a single set, the cache must also keep track of the address of the data actually contained in each set. The least significant bits of the address specify which set holds the data. The remaining most significant bits are called the tag and indicate which of the many possible addresses is held in that set.

8-entry x
(1+27+32)-bit
SRAM

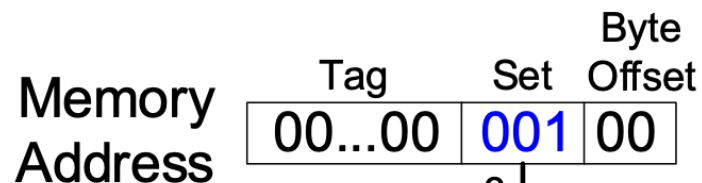


- Three bits are called the set bits because they indicate the set to which the address maps. (In general, the number of set bits is $\log_2 S$.)
- The remaining 27 tag bits indicate the memory address of the data stored in a given cache set.
- Sometimes, such as when the computer first starts up, the cache sets contain no data at all. The cache uses a valid bit for each set to indicate whether the set holds meaningful data. If the valid bit is 0, the contents are meaningless.

Direct Mapped Cache Hardware

RISC-V assembly code

```
    addi s0, zero, 5
    addi s1, zero, 0
LOOP: beq  s0, zero, DONE
      lw   s2, 4(s1)
      lw   s3, 12(s1)
      lw   s4, 8(s1)
      addi s0, s0, -1
      j    LOOP
DONE:
```



V Tag Data

0		
0		
0		
0		
1	00...00	mem[0x00...0C]
1	00...00	mem[0x00...08]
1	00...00	mem[0x00...04]
0		

Set 7 (111)

Set 6 (110)

Set 5 (101)

Set 4 (100)

Set 3 (011)

Set 2 (010)

Set 1 (001)

Set 0 (000)

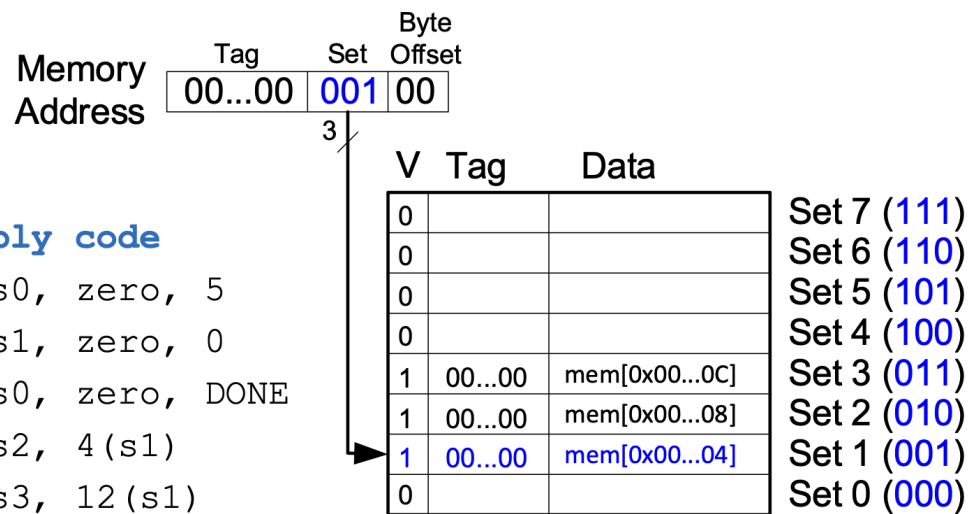
Miss Rate

Direct Mapped Cache Hardware

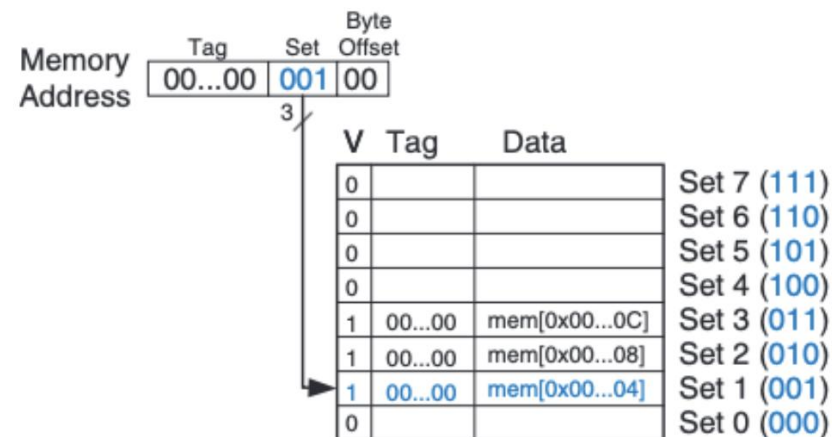
RISC-V assembly code

```
addi s0, zero, 5
addi s1, zero, 0
LOOP: beq s0, zero, DONE
      lw s2, 4(s1)
      lw s3, 12(s1)
      lw s4, 8(s1)
      addi s0, s0, -1
      j LOOP
```

DONE:

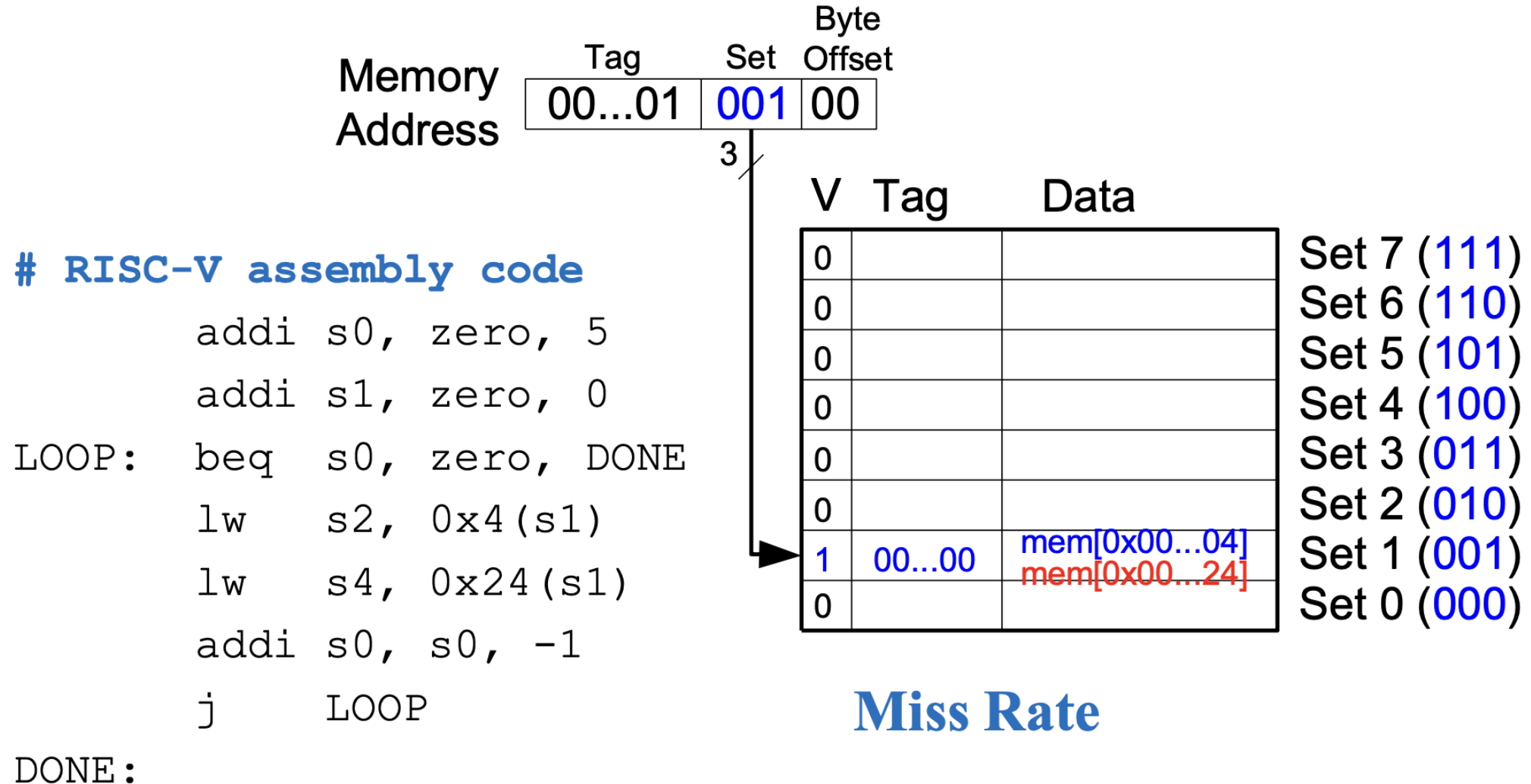


Miss Rate



Direct Mapped Cache: Conflict Miss

When two recently accessed addresses map to the same cache block, a conflict occurs, and the most recently accessed address evicts the previous one from the block. Direct mapped caches have only one block in each set, so two addresses that map to the same set always cause a conflict.



Direct Mapped Cache: Conflict Miss

When two recently accessed addresses map to the same cache block, a conflict occurs, and the most recently accessed address evicts the previous one from the block. Direct mapped caches have only one block in each set, so two addresses that map to the same set always cause a conflict.

